# BLUESTOCK
# IPO WEBSITE DESIGN

**Objective**: Develop a web application and REST API that provides IPO-related information to the public. The application will display information such as company logo, name, price band, opening and closing dates, issue size, issue type, listing date, status, IPO price, listing price, listing gain, current market price (CMP), and current return. Additionally, it will include downloadable RHP and DRHP PDFs.

## Software Requirements

### Backend:

● SDK: Python (Version 3.12.3)
● Framework: Django (Version 5.0.6) - `pip install Django`
● API: Django REST Framework (Version 3.15.1) - `pip install djangorestframework`
● Tools: Postman for API testing, Git & GitHub for version control

### Frontend:

● Technologies: HTML, CSS, plain JavaScript (no NodeJS)
● Framework: Bootstrap 5 (via CDN link)
● Database: PostgreSQL
● IDE: Visual Studio Code (VS Code)

## IPO(INITIAL PUBLIC OFFERING):

An Initial Public Offering (IPO) is a process where a private company offers shares of its stock to the public for the first time, making it publicly traded on the stock market. This allows the company to raise capital by selling shares to the public, and it also provides investors with an opportunity to own a part of the company.

**Example**:Imagine a company that has been privately held by a small group of investors. To raise more money, they decide to offer some of their ownership (shares) to the public for the first time. This "going public" process is the IPO. Investors can buy these shares, making them part-owners of the company and allowing the company to use the funds for growth.

**Django:**It is a high-level python web **framework** that simplifies and speeds up web application development.Django uses the Model-View-Template (MVT) architecture, which separates data management (Model), user interface logic (View), and presentation (Template).

A **Python framework** is a collection of pre-written code, tools, and libraries that provide a structure for developing software applications efficiently in Python. It saves you time by handling common tasks like routing, database access, authentication, and more.

**REST=REPRESENTATIONAL STATE TRANSFER**

A **REST API(Application Programming Interface)** is a way for different applications to **communicate with each other** over the internet using HTTP (like how your browser talks to websites).

It allows clients (like your frontend) to **request** or **send** data to/from a server (your backend) in a **structured format**, usually **JSON**.

Great! Since you're building a complete IPO Web App using Django, Django REST Framework, PostgreSQL, and a static frontend (HTML, CSS, JavaScript, Bootstrap), here is a full breakdown — with **clear instructions** — to help you **build the full website from scratch**:

---

# 🔧 PART 1: ENVIRONMENT SETUP

### ✅ 1. Install Required Tools (Only Once)

- Python 3.12.3
- PostgreSQL
- Visual Studio Code
- Git
- Postman

### ✅ 2. Create and Activate Virtual Environment

Open VS Code → Terminal:

```shell
python -m venv env
# Activate the environment
# Windows
env\Scripts\activate
# macOS/Linux
source env/bin/activate
```

### ✅ 3. Install Django, DRF, and PostgreSQL Adapter

```shell
pip install Django==5.0.6
pip install djangorestframework==3.15.1
pip install psycopg2
```

---

# 📁 PART 2: DJANGO PROJECT SETUP

### ✅ 4. Create Django Project and App

```shell
django-admin startproject ipo_project
cd ipo_project
python manage.py startapp ipo_app
```

### ✅ 5. Register Your App

Edit `ipo_project/settings.py`:

```python
INSTALLED_APPS = [
    ...
```

```
    'rest_framework',
    'ipo_app',
]
```

---

# 🗃️ PART 3: DATABASE CONFIGURATION

### ✅ 6. Setup PostgreSQL in `settings.py`

```python
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'ipo_db',
        'USER': 'your_username',
        'PASSWORD': 'your_password',
        'HOST': 'localhost',
        'PORT': '5432',
    }
}
```

Then create this database manually from pgAdmin or using SQL:

```sql
CREATE DATABASE ipo_db;
```

---

# 🧩 PART 4: CREATE IPO MODEL

### ✅ 7. Define IPO Schema

In `ipo_app/models.py`:

```python
from django.db import models

class IPO(models.Model):
    name = models.CharField(max_length=100)
    logo_url = models.URLField()
    price_band = models.CharField(max_length=50)
    open_date = models.DateField()
    close_date = models.DateField()
    issue_size = models.CharField(max_length=50)
    issue_type = models.CharField(max_length=50)
    listing_date = models.DateField()
    status = models.CharField(max_length=20)
    ipo_price = models.FloatField()
    listing_price = models.FloatField()
    listing_gain = models.FloatField()
    cmp = models.FloatField()
    current_return = models.FloatField()
    rhp_pdf = models.URLField()
    drhp_pdf = models.URLField()
```

### ✅ 8. Migrate Database

```shell
python manage.py makemigrations
python manage.py migrate
```

---

# 🔌 PART 5: CREATE API USING DRF

### ✅ 9. Create Serializer

`ipo_app/serializers.py`:

```python
from rest_framework import serializers
from .models import IPO

class IPOSerializer(serializers.ModelSerializer):
    class Meta:
        model = IPO
        fields = '__all__'
```

### ✅ 10. Create ViewSet

`ipo_app/views.py`:

```python
from rest_framework import viewsets
from .models import IPO
from .serializers import IPOSerializer

class IPOViewSet(viewsets.ModelViewSet):
    queryset = IPO.objects.all()
    serializer_class = IPOSerializer
```

### ✅ 11. Add API URLs

Create `ipo_app/urls.py`:

```python
from django.urls import path, include
from rest_framework.routers import DefaultRouter
from .views import IPOViewSet

router = DefaultRouter()
```

```
router.register(r'ipos', IPOViewSet)

urlpatterns = [
    path('', include(router.urls)),
]
```

Update `ipo_project/urls.py`:

```python
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include('ipo_app.urls')),
]
```

---

# ✅ PART 6: ADMIN PANEL FOR IPO DATA ENTRY

### ✅ 12. Register Model in Admin

`ipo_app/admin.py`:

```python
from django.contrib import admin
from .models import IPO

admin.site.register(IPO)
```

### ✅ 13. Create Superuser

```shell
python manage.py createsuperuser
```

Visit `http://localhost:8000/admin/` to log in and add IPO entries.

---

# 🎨 PART 7: FRONTEND DEVELOPMENT (STATIC)

### ✅ 14. Create Static `frontend/` Folder

Structure:

```unset
ipo_project/
├── frontend/
|   ├── index.html
|   └── script.js
```

### ✅ 15. index.html (Bootstrap)

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>IPO Web App</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css
/bootstrap.min.css" rel="stylesheet">
```

```
</head>
<body>
  <div class="container mt-4">
    <h2 class="mb-3">IPO Listings</h2>
    <div id="ipo-list"></div>
  </div>
  <script src="script.js"></script>
</body>
</html>
```

### ✅ 16. script.js (Fetch API)

JavaScript

```
fetch('http://localhost:8000/api/ipos/')
  .then(response => response.json())
  .then(data => {
    let output = "";
    data.forEach(ipo => {
      output += `
        <div class="card mb-3">
          <div class="card-body">
            <img src="${ipo.logo_url}" width="100" />
            <h5>${ipo.name}</h5>
            <p><strong>Price Band:</strong>
${ipo.price_band}<br>
            <strong>Dates:</strong> ${ipo.open_date} to
${ipo.close_date}<br>
            <strong>IPO Price:</strong> ₹${ipo.ipo_price},
<strong>CMP:</strong> ₹${ipo.cmp}<br>
            <strong>Return:</strong>
${ipo.current_return}%</p>
            <a href="${ipo.rhp_pdf}" class="btn
btn-primary">RHP</a>
```

```
            <a href="${ipo.drhp_pdf}" class="btn
btn-secondary">DRHP</a>
          </div>
        </div>`;
    });
    document.getElementById("ipo-list").innerHTML = output;
  });
```

To view the frontend, just open `index.html` in your browser.

---

# 🚀 PART 8: TESTING & DEPLOYMENT

### ✅ 17. Test API with Postman

- `GET http://localhost:8000/api/ipos/`
- `POST` with body to create new IPOs
- `PUT`, `PATCH`, `DELETE`

### ✅ 18. Use .env for Secrets

Use `python-decouple`:

```shell
pip install python-decouple
```

Then load secrets like DB password safely from `.env`.

### ✅ 19. Allow CORS (for frontend to talk to backend)

```shell
pip install django-cors-headers
```

Update `settings.py`:

```python
INSTALLED_APPS += ['corsheaders']
MIDDLEWARE = ['corsheaders.middleware.CorsMiddleware'] +
MIDDLEWARE
CORS_ALLOW_ALL_ORIGINS = True
```

---

# 📦 PART 9: DEPLOYMENT OPTIONS

**You can deploy on:**

- **Render.com**
- **Railway.app**
- **Heroku**

Let me know if you need a deployment guide too.

---

# 📄 PART 10: DOCUMENTATION

### ✅ 20. Create `README.md` with:

- Project description
- Setup instructions

- API endpoints
- Screenshots
- Team contributions